

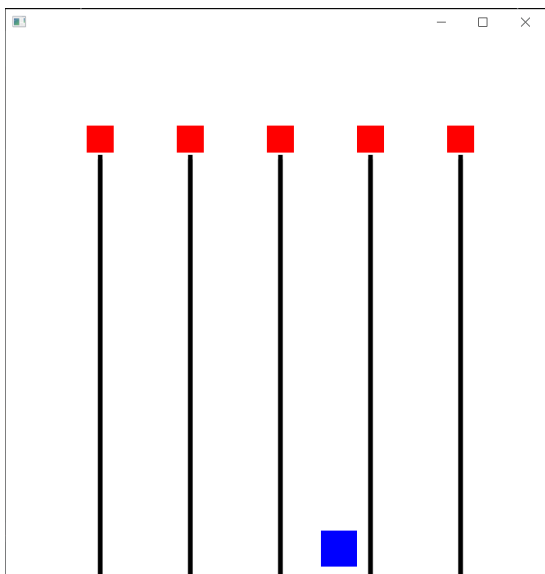
Practice 9

Deadline: 2 weeks from now. Should be checked onsite (during labs).

Task 1 - Space Invader Game

In the space invader game, the player is the blue rectangle, which could move to four direction by typing the corresponding arrow keys on your keyboard. In addition, by pressing **space**, the player could shoot bullets to the red enemies. Yet, the red enemies could also shoot bullets to the player. Whoever gets shot will disappear (the bullet also disappears).

Unfortunately, `SpaceInvaderGame.java` has some obvious flaws: enemies bullets keep coming while the player cannot shoot any bullet.



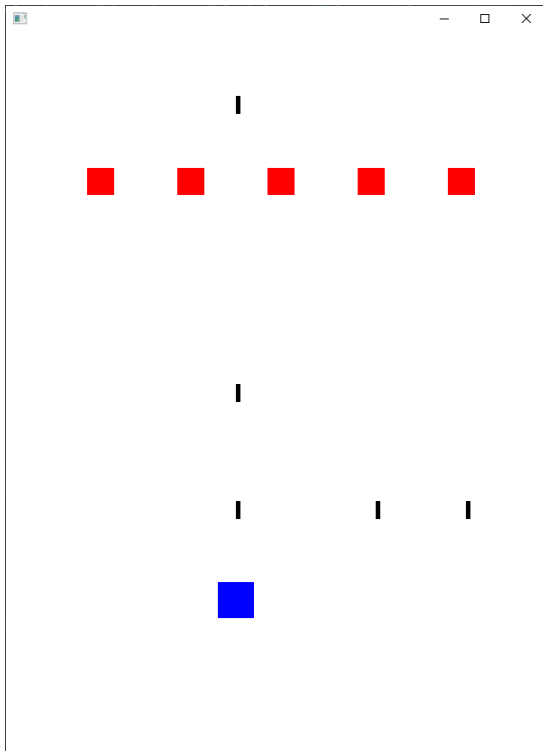
Your task is to fix `SpaceInvaderGame.java` so that:

- The player can shoot bullet to enemies, which will disappear when got shot.
- The enemies should shoot bullets with certain random intervals.

Note that JavaFX's `AnimationTimer` allows us to create a timer that is called in each **frame** while it is active. Since `AnimationTimer` is an abstract class; we need to create a custom class which extends it and override its `handle` method, **which is called in every frame**. `AnimationTimer`'s `start()` method starts the timer and the `stop()` method stops it.

Typically, the Animation Timer's speed is ~60 fps (frames per second), meaning that in every second, frames will be rendered ~60 times, therefore `handle` method will be invoked ~60 times. That's why we observe the animation, which is in fact different frames being rendered fast.

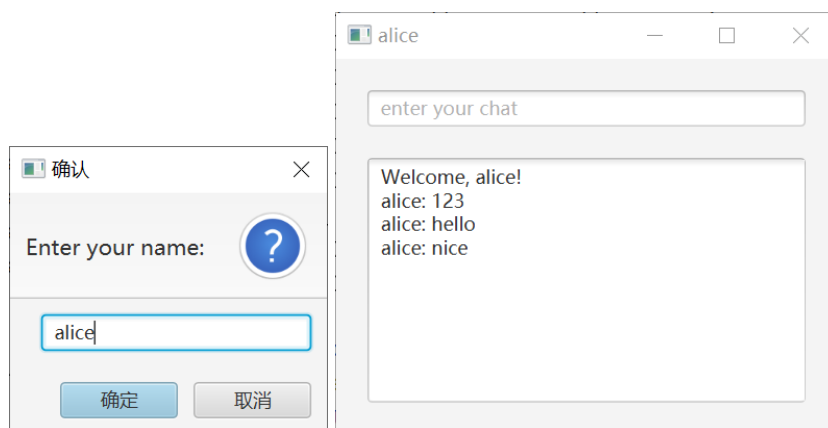
The expected result:



Task 2 - Broadcast Chat

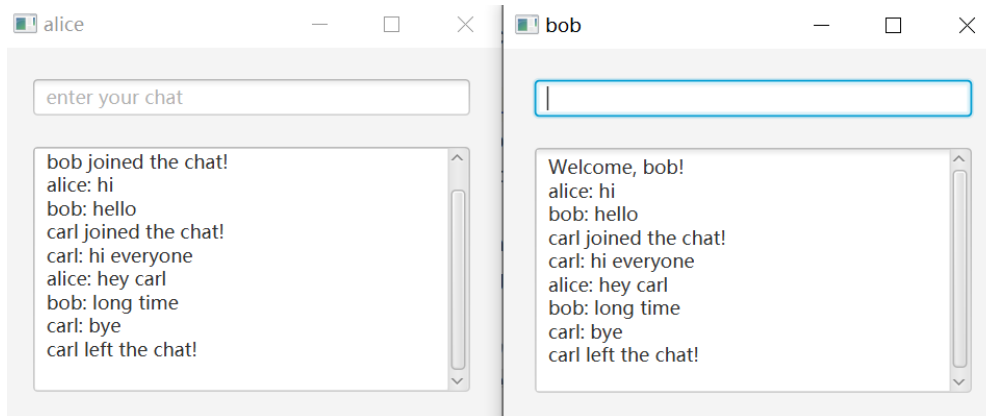
Last week, we created a simple C/S broadcasting application using Sockets and Multithreading in [Tutorial-Socket.pdf](#). In this task, we'll turn the broadcast console program to a GUI program with JavaFX.

To get you started, we provide an incomplete implementation that allows user to input name, type anything in the text field, press Enter, then display the input in the text area. Download the [BroadcastFX.zip](#) from BB to access the code.



Note that, the controller class can define an `initialize()` method, which is called after all `@FXML` annotated fields have been injected. Basically, in the controller class, the constructor is called first, then any `@FXML` annotated fields are populated, then `initialize()` is called. So, the constructor does not have access to `@FXML` fields, while `initialize()` does have access to them.

Your task is to turn this code into a C/S broadcast GUI program as in the tutorial. Multiple clients could connect to the server, and send messages to the server, which will broadcast the messages to all clients.



Tips

- You may reuse the `ChatServer` from our tutorial, which is also available on BB.
- You may update `ChatClientController` to add socket communication and multi-threading.
- You may use `Platform.runLater()` to update the GUI from a non-GUI thread.
- You may use `stage.setOnCloseRequest(...)` to handle the event when clients click X to close the window.

Evaluation

The practice will be checked by teachers or SAs. What will be tested:

1. That you understand every line of your own code, not just copy from somewhere
2. That your program compiles correctly (javac)
3. Correctness of the program logic
4. That the result is obtained in a reasonable time

Late submissions after the deadline will incur a 20% penalty, meaning that you can only get 80% of this practice's score.